COMP102 Lecture 5: Binary Trees

Structure, Terminology, Depth/Height, and Traversals

COMP102 Teaching Team
April 21, 2026

Plan

1 Learning Outcomes
2 Motivation
3 Binary Trees
4 Representation
5 Depth and Height
6 DFS Traversals
7 BFS Traversal

Learning Outcomes

Learning outcomes

By the end of this lecture, students should be able to:

- define a **binary tree** and identify its main parts (root, children, leaves, subtrees);
- use standard **tree terminology** (parent/child/sibling/ancestor/descendant);
- compute **depth** and **height** and explain the difference;
- implement a minimal `Node` class and build small trees;
- trace **preorder**, **inorder**, and **postorder** traversals (DFS);
- trace **breadth-first** traversal (BFS) and explain why it uses a queue.

Trees are the first major shift away from **linear** data. The goal is to understand **branching structure** and how different traversals *read* that structure.

Motivation

Why a new structure?

Most structures we have seen are fundamentally **linear**: there is always a "next element."

Why a new structure?

Most structures we have seen are fundamentally **linear**: there is always a "next element."

Linear containers

- arrays / lists
- stacks (LIFO)
- queues (FIFO)
- linked lists

Why a new structure?

Most structures we have seen are fundamentally **linear**: there is always a "next element."

Linear containers

- arrays / lists
- stacks (LIFO)
- queues (FIFO)
- linked lists

Branching situations

- folders & files
- tournament brackets
- decision processes
- arithmetic expressions

Why a new structure?

Most structures we have seen are fundamentally **linear**: there is always a "next element."

Linear containers

- arrays / lists
- stacks (LIFO)
- queues (FIFO)
- linked lists

Branching situations

- folders & files
- tournament brackets
- decision processes
- arithmetic expressions

A tree is what happens when data no longer lives in a line: **it branches**.

Motivation: expressions have structure

Consider the expression:

$$(3 + 5) \times (10 - 2)$$

Motivation: expressions have structure

Consider the expression:

$$(3 + 5) \times (10 - 2)$$

This is not just a bag of numbers. It has a **hierarchy**:

Motivation: expressions have structure

Consider the expression:

$$(3 + 5) \times (10 - 2)$$

This is not just a bag of numbers. It has a **hierarchy**:

- the final operation is multiplication;

Motivation: expressions have structure

Consider the expression:

$$(3 + 5) \times (10 - 2)$$

This is not just a bag of numbers. It has a **hierarchy**:

- the final operation is multiplication;
- it combines two subexpressions;

Motivation: expressions have structure

Consider the expression:

$$(3 + 5) \times (10 - 2)$$

This is not just a bag of numbers. It has a **hierarchy**:

- the final operation is multiplication;
- it combines two subexpressions;
- each subexpression is itself built from smaller parts.

Motivation: expressions have structure

Consider the expression:

$$(3 + 5) \times (10 - 2)$$

This is not just a bag of numbers. It has a **hierarchy**:

- the final operation is multiplication;
- it combines two subexpressions;
- each subexpression is itself built from smaller parts.

Remark 2.6. Guiding question

How do we store the **shape of the computation**, not just the final answer?

Another representation

Another representation

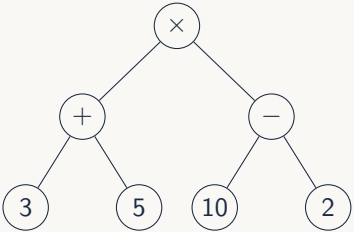
Expression

$$(3 + 5) \times (10 - 2)$$

Another representation

Expression

$$(3 + 5) \times (10 - 2)$$



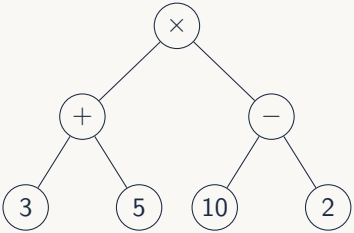
Another representation

Expression

$$(3 + 5) \times (10 - 2)$$

Interpretation

- internal nodes: operators
- leaves: values
- each operator has (up to) two inputs



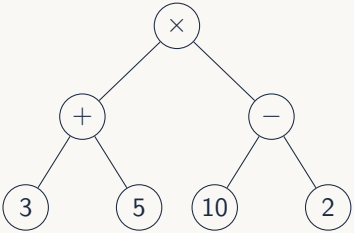
Another representation

Expression

$$(3 + 5) \times (10 - 2)$$

Interpretation

- internal nodes: operators
- leaves: values
- each operator has (up to) two inputs



This tree is a **map of dependencies**: you cannot compute a parent before understanding its children.

Binary Trees

Tree vs binary tree

Tree vs binary tree

Tree (informal)

A hierarchical structure with a distinguished **root** node and **children** branching downward.

Tree vs binary tree

Tree (informal)

A hierarchical structure with a distinguished **root** node and **children** branching downward.

Binary tree

A tree where each node has **at most two children**, conventionally called **left** and **right**.

Tree vs binary tree

Tree (informal)

A hierarchical structure with a distinguished **root** node and **children** branching downward.

Binary tree

A tree where each node has **at most two children**, conventionally called **left** and **right**.

Example 3.4. What "binary" does *not* mean

It does **not** mean the values are ordered. A binary tree is about **shape** (two child pointers), not about sortedness.

Tree vs binary tree

Tree (informal)

A hierarchical structure with a distinguished **root** node and **children** branching downward.

Binary tree

A tree where each node has **at most two children**, conventionally called **left** and **right**.

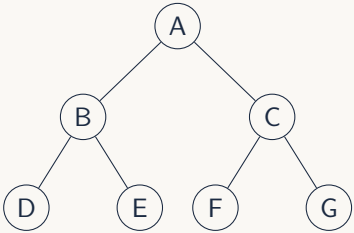
Example 3.5. What "binary" does *not* mean

It does **not** mean the values are ordered. A binary tree is about **shape** (two child pointers), not about sortedness.

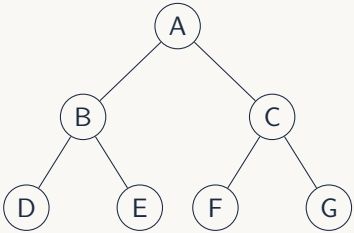
Three possibilities

- no children (leaf)
- one child
- two children

Terminology on one picture

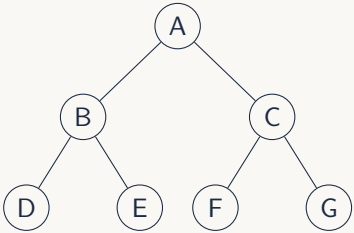


Terminology on one picture



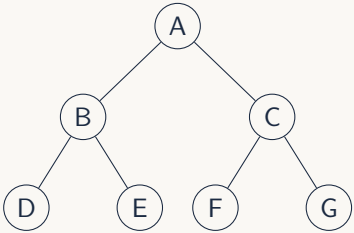
- **root**: A

Terminology on one picture



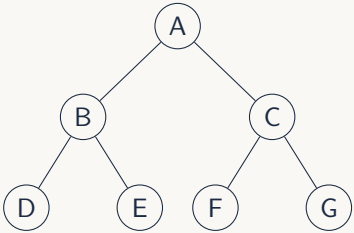
- **root**: A
- **children of A**: B, C

Terminology on one picture



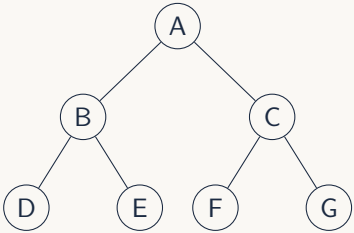
- **root**: A
- **children of A**: B, C
- **parent of E**: B

Terminology on one picture



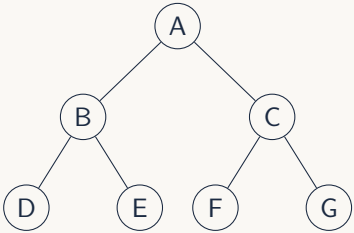
- **root**: A
- **children of A**: B, C
- **parent of E**: B
- **siblings**: D and E

Terminology on one picture



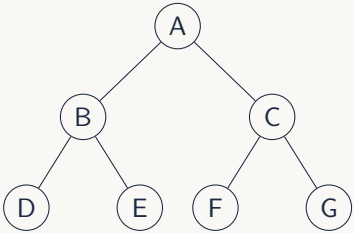
- **root**: A
- **children of A**: B, C
- **parent of E**: B
- **siblings**: D and E
- **leaves**: D, E, F, G

Terminology on one picture



- **root**: A
- **children of A**: B, C
- **parent of E**: B
- **siblings**: D and E
- **leaves**: D, E, F, G
- **subtree at B**: (B, D, E)

Terminology on one picture



- **root**: A
- **children of A**: B, C
- **parent of E**: B
- **siblings**: D and E
- **leaves**: D, E, F, G
- **subtree at B**: (B, D, E)
- **ancestor/descendant**: A is ancestor of all

Representation

A minimal node class

A simple implementation stores:

A minimal node class

A simple implementation stores:

- a value

A minimal node class

A simple implementation stores:

- a value
- a pointer to the left child

A minimal node class

A simple implementation stores:

- a value
- a pointer to the left child
- a pointer to the right child

A minimal node class

A simple implementation stores:

- a value
- a pointer to the left child
- a pointer to the right child

Python `Node`

```python
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
```

A minimal node class

A simple implementation stores:

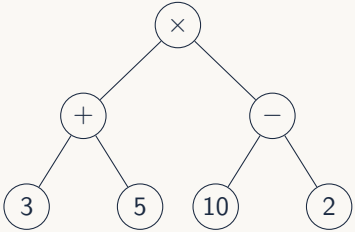
- a value
- a pointer to the left child
- a pointer to the right child

Python `Node`

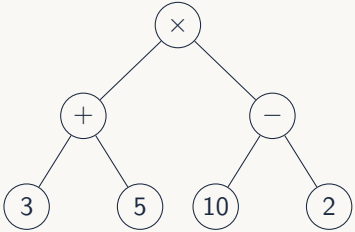
```python
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
```

In many basic implementations, the **entire tree** is represented by a single reference: **the root**.

Building a tree by linking nodes



Building a tree by linking nodes



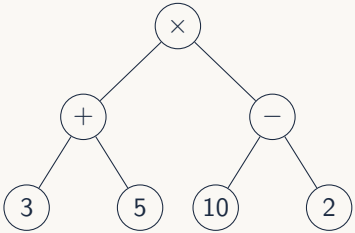
Building a tree by linking nodes

Construct the expression tree

```
root = Node('*')
root.left = Node('+')
root.right = Node('-')

root.left.left = Node(3)
root.left.right = Node(5)

root.right.left = Node(10)
root.right.right = Node(2)
```

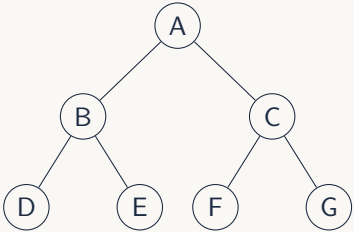


Depth and Height

Depth: distance from the root

Definition

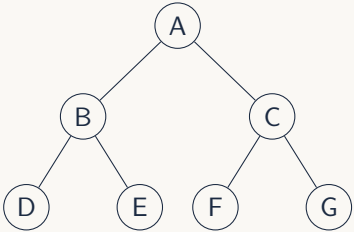
The **depth** of a node is the number of edges on the path from the **root** to that node.



Depth: distance from the root

Definition

The **depth** of a node is the number of edges on the path from the **root** to that node.

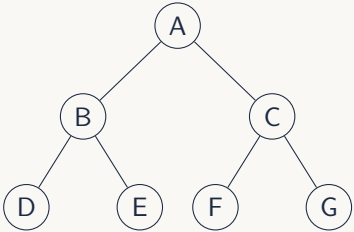


- $\text{depth}(A) = 0$

Depth: distance from the root

Definition

The **depth** of a node is the number of edges on the path from the **root** to that node.

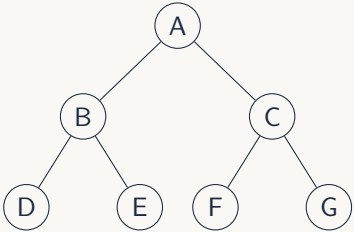


- $\text{depth}(A) = 0$
- $\text{depth}(B) = 1$

Depth: distance from the root

Definition

The **depth** of a node is the number of edges on the path from the **root** to that node.

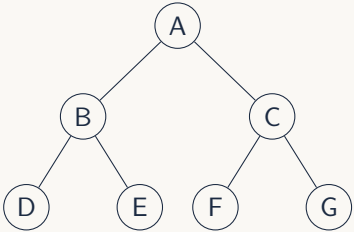


- $\text{depth}(A) = 0$
- $\text{depth}(B) = 1$
- $\text{depth}(E) = 2$

Depth: distance from the root

Definition

The **depth** of a node is the number of edges on the path from the **root** to that node.



- $\text{depth}(A) = 0$
- $\text{depth}(B) = 1$
- $\text{depth}(E) = 2$

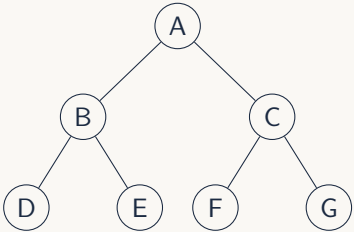
Remark 5.5. Direction

Depth is measured **from the top**.

Height: distance to the deepest leaf

Definition

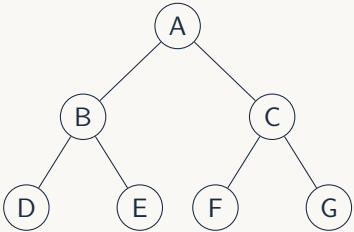
The **height** of a node is the number of edges on the longest path from that node down to any leaf.



Height: distance to the deepest leaf

Definition

The **height** of a node is the number of edges on the longest path from that node down to any leaf.

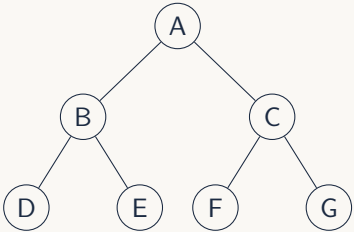


- height(D) = 0 (leaf)

Height: distance to the deepest leaf

Definition

The **height** of a node is the number of edges on the longest path from that node down to any leaf.

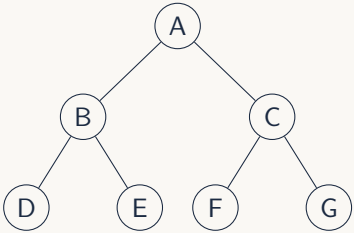


- height(D) = 0 (leaf)
- height(B) = 1

Height: distance to the deepest leaf

Definition

The **height** of a node is the number of edges on the longest path from that node down to any leaf.

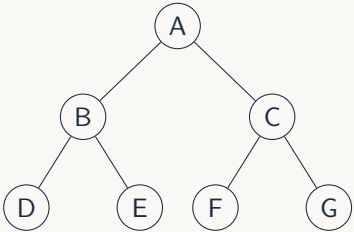


- height(D) = 0 (leaf)
- height(B) = 1
- height(A) = 2 (tree height)

Height: distance to the deepest leaf

Definition

The **height** of a node is the number of edges on the longest path from that node down to any leaf.



- height(D) = 0 (leaf)
- height(B) = 1
- height(A) = 2 (tree height)

Remark 5.10. Direction

Height is measured **from the bottom**.

Depth vs height

Depth

- how far from the root?
- measured downward from the top
- root depth $= 0$

Height

- how far to the deepest leaf?
- measured downward from the node
- leaf height $= 0$

Depth answers "*where am I?*" Height answers "*how much is below me?*"

DFS Traversals

Traversal: reading a tree

A traversal is a rule for visiting every node in some order.

Two big families

Traversal: reading a tree

A traversal is a rule for visiting every node in some order.

Two big families

- **DFS**: go deep along a branch before moving sideways

Traversal: reading a tree

A traversal is a rule for visiting every node in some order.

Two big families

- **DFS**: go deep along a branch before moving sideways
- **BFS**: visit level by level

Three moments to “visit” a node

When doing a recursive DFS at a node, there are three natural moments:

Three moments to “visit” a node

When doing a recursive DFS at a node, there are three natural moments:

1. **before** exploring children

Three moments to “visit” a node

When doing a recursive DFS at a node, there are three natural moments:

1. **before** exploring children
2. **between** exploring left and right

Three moments to “visit” a node

When doing a recursive DFS at a node, there are three natural moments:

1. **before** exploring children
2. **between** exploring left and right
3. **after** exploring both children

Three moments to “visit” a node

When doing a recursive DFS at a node, there are three natural moments:

1. **before** exploring children
2. **between** exploring left and right
3. **after** exploring both children

These correspond to:

Three moments to “visit” a node

When doing a recursive DFS at a node, there are three natural moments:

1. **before** exploring children
2. **between** exploring left and right
3. **after** exploring both children

These correspond to:

- preorder: Node, Left, Right

Three moments to “visit” a node

When doing a recursive DFS at a node, there are three natural moments:

1. **before** exploring children
2. **between** exploring left and right
3. **after** exploring both children

These correspond to:

- preorder: Node, Left, Right
- inorder: Left, Node, Right

Three moments to “visit” a node

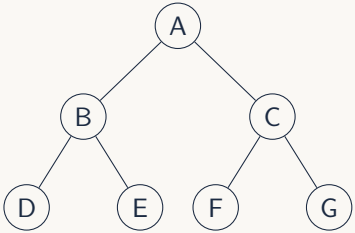
When doing a recursive DFS at a node, there are three natural moments:

1. **before** exploring children
2. **between** exploring left and right
3. **after** exploring both children

These correspond to:

- preorder: Node, Left, Right
- inorder: Left, Node, Right
- postorder: Left, Right, Node

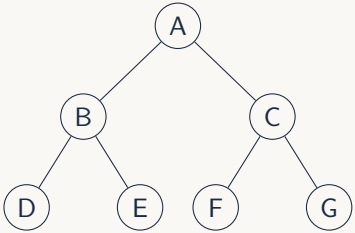
Preorder (Node, Left, Right)



Preorder (Node, Left, Right)

Recursive idea

Visit the node **first**, then traverse the left subtree, then the right.



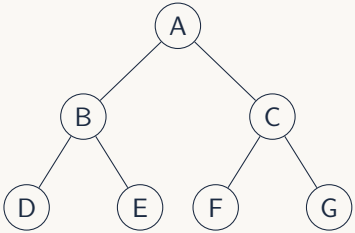
Preorder (Node, Left, Right)

Recursive idea

Visit the node **first**, then traverse the left subtree, then the right.

Code

```
def preorder(node):
    if node is None:
        return
    visit(node)
    preorder(node.left)
    preorder(node.right)
```



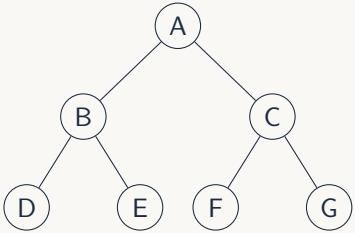
Preorder (Node, Left, Right)

Recursive idea

Visit the node **first**, then traverse the left subtree, then the right.

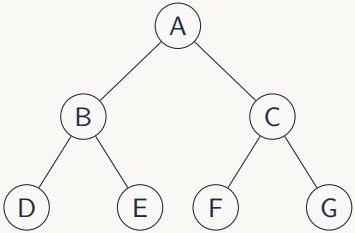
Code

```python
def preorder(node):
    if node is None:
        return
    visit(node)
    preorder(node.left)
    preorder(node.right)
```



Output: A, B, D, E, C, F, G

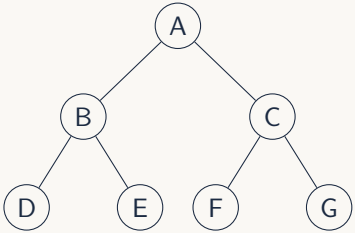
Inorder (Left, Node, Right)



Inorder (Left, Node, Right)

Recursive idea

Traverse left subtree, then visit the node, then traverse right subtree.



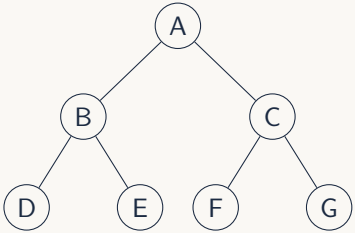
Inorder (Left, Node, Right)

Recursive idea

Traverse left subtree, then visit the node, then traverse right subtree.

Code

```
def inorder(node):
    if node is None:
        return
    inorder(node.left)
    visit(node)
    inorder(node.right)
```



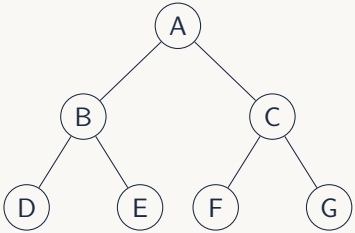
Inorder (Left, Node, Right)

Recursive idea

Traverse left subtree, then visit the node, then traverse right subtree.

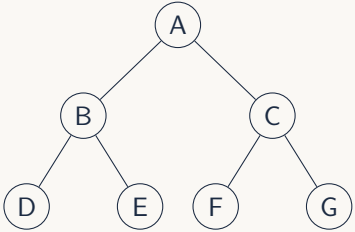
Code

```
def inorder(node):
    if node is None:
        return
    inorder(node.left)
    visit(node)
    inorder(node.right)
```



Output: D, B, E, A, F, C, G

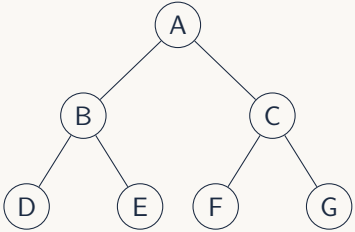
Postorder (Left, Right, Node)



Postorder (Left, Right, Node)

Recursive idea

Traverse children **first**; visit the node only after both subtrees.



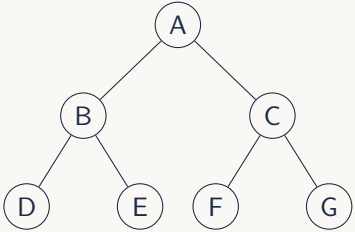
Postorder (Left, Right, Node)

Recursive idea

Traverse children **first**; visit the node only after both subtrees.

Code

```
def postorder(node):
    if node is None:
        return
    postorder(node.left)
    postorder(node.right)
    visit(node)
```



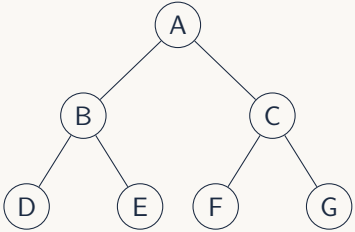
Postorder (Left, Right, Node)

Recursive idea

Traverse children **first**; visit the node only after both subtrees.

Code

```python
def postorder(node):
    if node is None:
        return
    postorder(node.left)
    postorder(node.right)
    visit(node)
```

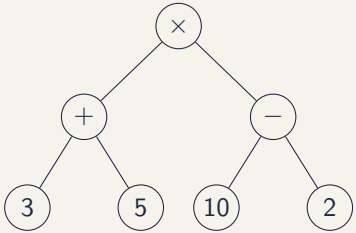


Output: D, E, B, F, G, C, A

Traversals and expression notation

For expression trees, the three DFS orders correspond to three classic notations.

Same tree

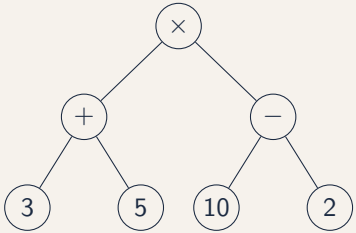


Three readings

Traversals and expression notation

For expression trees, the three DFS orders correspond to three classic notations.

Same tree



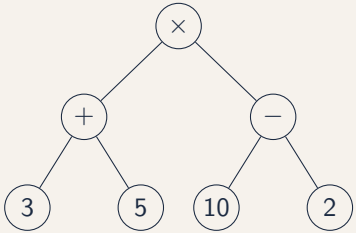
Three readings

- preorder (prefix): $\times\ +\ 3\ 5\ -\ 10\ 2$

Traversals and expression notation

For expression trees, the three DFS orders correspond to three classic notations.

Same tree



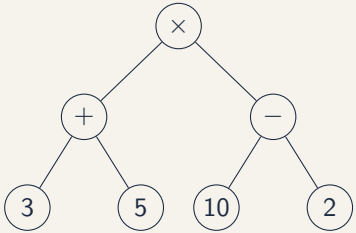
Three readings

- preorder (prefix): $\times\ +\ 3\ 5\ -\ 10\ 2$
- inorder (infix): $(3 + 5) \times (10 - 2)$

Traversals and expression notation

For expression trees, the three DFS orders correspond to three classic notations.

Same tree



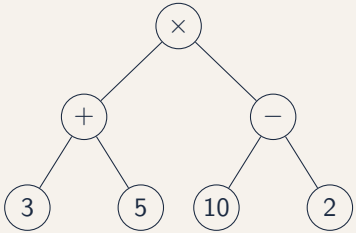
Three readings

- preorder (prefix): $\times\ +\ 3\ 5\ -\ 10\ 2$
- inorder (infix): $(3 + 5) \times (10 - 2)$
- postorder (postfix): $3\ 5\ +\ 10\ 2\ -\ \times$

Traversals and expression notation

For expression trees, the three DFS orders correspond to three classic notations.

Same tree



Three readings

- preorder (prefix): $\times\ +\ 3\ 5\ -\ 10\ 2$
- inorder (infix): $(3 + 5) \times (10 - 2)$
- postorder (postfix): $3\ 5\ +\ 10\ 2\ -\ \times$

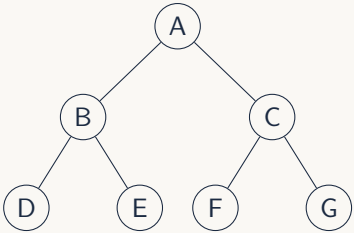
These traversals are not arbitrary: they are different, useful ways to **serialize** the same structure.

BFS Traversal

BFS: level-order traversal

Idea

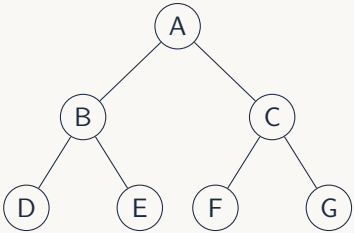
Breadth-first traversal visits nodes **level by level**: first the root, then all depth-1 nodes, then all depth-2 nodes, etc.



BFS: level-order traversal

Idea

Breadth-first traversal visits nodes **level by level**: first the root, then all depth-1 nodes, then all depth-2 nodes, etc.



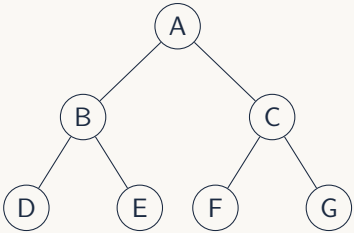
Output

A, B, C, D, E, F, G

BFS: level-order traversal

Idea

Breadth-first traversal visits nodes **level by level**: first the root, then all depth-1 nodes, then all depth-2 nodes, etc.



Output

A, B, C, D, E, F, G

Remark 7.3. Different philosophy

DFS dives into a branch. BFS expands outward.

Why BFS uses a queue

To visit nodes in the order they are discovered, BFS keeps a **frontier** of "next nodes to process."

Queue discipline

Why BFS uses a queue

To visit nodes in the order they are discovered, BFS keeps a **frontier** of "next nodes to process."

Queue discipline

- nodes discovered earlier should be processed earlier (FIFO)

Why BFS uses a queue

To visit nodes in the order they are discovered, BFS keeps a **frontier** of "next nodes to process."

Queue discipline

- nodes discovered earlier should be processed earlier (FIFO)
- children are added to the back as we process parents

Why BFS uses a queue

To visit nodes in the order they are discovered, BFS keeps a **frontier** of "next nodes to process."

Queue discipline

- nodes discovered earlier should be processed earlier (FIFO)
- children are added to the back as we process parents

BFS is a natural application of queues: it repeatedly processes the oldest waiting node.

BFS in Python

Typical implementation

```python
from collections import deque

def bfs(root):
    if root is None:
        return
    q = deque([root])
    while q:
        node = q.popleft()
        visit(node)
        if node.left is not None:
            q.append(node.left)
        if node.right is not None:
            q.append(node.right)
```

BFS in Python

Typical implementation

```python
from collections import deque

def bfs(root):
    if root is None:
        return
    q = deque([root])
    while q:
        node = q.popleft()
        visit(node)
        if node.left is not None:
            q.append(node.left)
        if node.right is not None:
            q.append(node.right)
```

Example 7.2. Mental model

"Process the front; append newly discovered children to the back."

Wrap-up

DFS vs BFS: side-by-side

Feature	DFS	BFS
Exploration	go deep along a branch	go level by level
Typical tool	recursion / stack	queue
Common outputs	preorder / inorder / postorder	level order
Good for	structure, recursion patterns	layered exploration; shortest paths later

Traversals are not “extra tricks.” They are the main way we interact with trees.

Takeaways

Takeaways

- A binary tree models **branching hierarchy**.

Takeaways

- A binary tree models **branching hierarchy**.
- Tree terminology is about relationships: parent/child/leaf/subtree.

Takeaways

- A binary tree models **branching hierarchy**.
- Tree terminology is about relationships: parent/child/leaf/subtree.
- Depth and height measure structure from **different directions**.

Takeaways

- A binary tree models **branching hierarchy**.
- Tree terminology is about relationships: parent/child/leaf/subtree.
- Depth and height measure structure from **different directions**.
- DFS traversals differ only by **when the node is visited**.

Takeaways

- A binary tree models **branching hierarchy**.
- Tree terminology is about relationships: parent/child/leaf/subtree.
- Depth and height measure structure from **different directions**.
- DFS traversals differ only by **when the node is visited**.
- BFS is level-order and naturally uses a **queue**.

Takeaways

- A binary tree models **branching hierarchy**.
- Tree terminology is about relationships: parent/child/leaf/subtree.
- Depth and height measure structure from **different directions**.
- DFS traversals differ only by **when the node is visited**.
- BFS is level-order and naturally uses a **queue**.

Once you can draw a tree and trace traversals, you can reason about most tree algorithms.

Preview: binary search trees (next lecture)

Today, a binary tree was only a **shape** (up to two children).

Next lecture we will add a rule about the **values**:

Preview: binary search trees (next lecture)

Today, a binary tree was only a **shape** (up to two children).

Next lecture we will add a rule about the **values**:

Teaser

What if every node obeys an ordering constraint between its left and right subtrees?

Preview: binary search trees (next lecture)

Today, a binary tree was only a **shape** (up to two children).

Next lecture we will add a rule about the **values**:

Teaser

What if every node obeys an ordering constraint between its left and right subtrees?

Remark 8.3. What changes

The same traversal ideas still apply, but the ordering rule enables efficient **search**.